
Team 5 Final Report**Comparative Study of Semiconductor Process Scheduling Algorithms Considering Q-time, Preventive Maintenance, and Stochastic Equipment Failures**

Park Seonghyun¹, Kim Hyunghoon², Kim Jinho³, Seo Hoon⁴, Lee Seungmin⁵

¹Department of Industrial and Information Systems Engineering, Soongsil University 20232808

²Department of Industrial and Information Systems Engineering, Soongsil University 20192208

³Department of Industrial and Information Systems Engineering, Soongsil University 20211275

⁴Department of Industrial and Information Systems Engineering, Soongsil University 20211286

⁵Department of Industrial and Information Systems Engineering, Soongsil University 20211296

Abstract This study addresses a semiconductor process scheduling problem that includes Q-time constraints, stochastic equipment failures, and preventive maintenance (PM). Based on the SMT_2020 testbed, product routes, equipment groups, setup, Q-time, failure, and PM parameters are reconstructed, and several scheduling approaches are compared in a SimPy-based discrete-event simulation environment. Basic dispatching rules, a Composite rule, a PM rule, and Genetic Algorithm-based policies are evaluated using the same input data and simulator, and performance is measured by a normalized objective function that jointly reflects makespan and average Q-time violation. This report summarizes the implementation method and evaluation criteria of each approach and analyzes the trade-off between production efficiency and Q-time constraint management. The experimental results show that the Composite rule, a rule-based method that combines several priority terms, achieved a better balance between makespan and average Q-time violation than single-criterion rules, and it reduced failures during processing while maintaining this balance when combined with PM. In contrast, GA reduced average Q-time violation further by jointly exploring job sequences, machine assignments, and PM thresholds, but its computation time was substantially longer, making it closer to a pre-optimization method or benchmark than to a real-time dispatching rule.

Keywords semiconductor process, scheduling, Q-time, preventive maintenance, equipment failure, Dispatching rule, Genetic Algorithm

1 Introduction

In semiconductor manufacturing processes, multiple jobs occupy equipment and are processed according to predefined process sequences. Since assigning limited equipment to specific jobs in specific orders directly affects due-date compliance, equipment utilization, and production lead time, scheduling decisions that jointly determine job sequences and equipment assignments are important.

A semiconductor process is difficult to view simply as a problem of determining the processing order of jobs. A single job is processed through multiple pieces of equipment according to a predefined process sequence, and certain operations can be performed on multiple alternative machines. In addition, even for the same operation, pro-

cessing time differs depending on the selected machine, so both job sequencing and machine assignment must be determined together. This structure is closely related to the characteristics of the Flexible Job Shop Scheduling Problem (FJSP), which includes the selection of alternative machines for each operation (Brandimarte, 1993).

However, in real manufacturing environments, operational characteristics beyond job sequencing and machine assignment must also be considered. Semiconductor processes include Q-time constraints, where the next operation must start within a certain time after one operation is completed; exceeding this limit can lead not merely to schedule delay but also to quality degradation, and depending on the degree of violation, rework or scrap may occur (Cho et al., 2014). In addition, equipment may be deliberately taken offline for preventive maintenance.

nance (PM) or interrupted by unexpected failures (Al-Hinai & ElMekkawy, 2011; Cassady & Kutanoglu, 2005). Therefore, semiconductor process scheduling can be viewed as a dynamic decision-making problem in which quality constraints such as Q-time, planned downtime such as PM, and stochastic uncertainty from equipment failures act simultaneously.

In such an environment, it is difficult to evaluate scheduling performance using only the overall job completion time, makespan. Even if a particular job is processed quickly, prolonged waiting by other jobs may increase Q-time violation; conversely, prioritizing certain jobs to reduce Q-time violation may increase the overall completion time. As a result, a balance between production efficiency and quality constraint management is required, and a decision criterion that jointly reflects makespan and Q-time violation is needed.

Related prior studies have treated the FJSP structure, Q-time constraints, preventive maintenance, and equipment failures as important scheduling elements (Al-Hinai & ElMekkawy, 2011; Brandimarte, 1993; Cassady & Kutanoglu, 2005; Cho et al., 2014). For example, studies have presented FJSP models considering alternative machine selection by operation (Brandimarte, 1993), modeled inter-operation waiting time constraints, and linked preventive maintenance with production schedules (Cassady & Kutanoglu, 2005; Cho et al., 2014). Flexible job shop scheduling problems with equipment failures have also been addressed (Al-Hinai & ElMekkawy, 2011). These studies provide key elements connected to the problem setting of this study, but they differ from a structure that evaluates Q-time violation, availability changes caused by equipment failures, and preventive maintenance decisions together in a single simulation environment.

This study considers Q-time constraints, stochastic equipment failures, and PM together in a single simulation environment. As a baseline approach, a rule-based algorithm is constructed, and a Genetic Algorithm is also applied to explore a broader solution space (Pezzella et al., 2008). The makespan and Q-time violation of the two approaches are then compared, and the trade-off between production completion time and inter-operation time constraints is analyzed.

2 Problem Setting and Simulation Configuration

2.1 Data

This study uses a dataset reconstructed for this problem from the semiconductor manufacturing testbed SMT_2020 (Semiconductor Manufacturing Testbed 2020). The dataset consists of 60 jobs and 60 machines. The jobs are divided into two product types, P1 and P2, with 30 jobs each, and the machines are divided into five groups, G1–G5.

figure 2.1.1 shows the process routes by product type and the Q-time limits of specific operations. P1 consists of 14 operations and P2 consists of 15 operations, and the same equipment group is repeatedly used at multiple operation stages. Each operation can be processed on any machine within the specified equipment group.

table 2.1.1 shows the input parameters by equipment group. Processing times follow values drawn from different uniform distributions for each product type, and setup occurs only when a machine switches to processing a different product type. PM time has a shorter value than post-failure repair time.

2.2 Objective Function

The performance criterion is defined as a normalized objective function that jointly reflects makespan and average Q-time violation. We design an algorithm that finds a schedule minimizing the following objective function among all schedules $\sigma \in \Sigma$ that satisfy the operational constraints of the simulation environment.

$$\min_{\sigma \in \Sigma} J(\sigma) = w_1 \frac{C_{\max}}{mk_{\text{ref}}} + w_2 \frac{AQT}{qv_{\text{ref}}}$$

$$C_{\max} = \max_{i \in \mathcal{J}} C_i, \quad AQT = \frac{1}{N} \sum_{i=1}^N v_i$$

$\sigma \in \Sigma$ Schedule satisfying operational constraints

C_i Completion time of job i

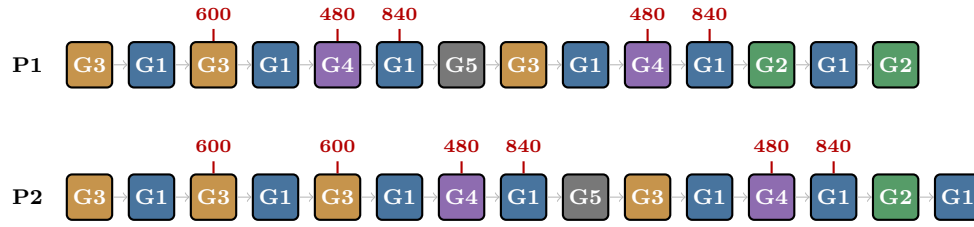
v_i Total Q-time violation of job i (min)

N Number of jobs (= 60)

$mk_{\text{ref}}, qv_{\text{ref}}$ Normalization reference values (maximum observed baseline)

w_1, w_2 Weights ($w_1 = w_2 = 1$)

Although makespan and AQT have the same unit, their observed ranges differ, so adding them without normalization would cause the larger-scale makespan term to dominate J . To prevent this, the maximum values observed



Colored boxes indicate machine groups G1–G5. Red labels = Q-time limit (min).

figure 2.1.1: Process routes by product type

when repeatedly running the basic dispatching rules with the same seed ($mk_{\text{ref}} \approx 14,734$ min, $qv_{\text{ref}} \approx 1,140$ min) are used as reference values, converting each term into a ratio relative to the baseline. The two indicators are assigned equal weights.

2.3 Simulation Environment

This simulation was implemented as a discrete-event simulation based on Python’s SimPy library. This section explains how input data are handled as state changes and events within the simulation, and how makespan and Q-time violation are calculated as a result. The description proceeds in the following order: the operation flow from the job perspective, equipment state changes and failure/repair/PM handling, and Q-time violation calculation.

2.3.1 Job Operation Flow

figure 2.3.1 shows the operation flow of jobs. All jobs are released at time 0, have identical job priority, and machine-to-machine transfer time is assumed to be 0. Before assignment to equipment, each job always passes through a waiting area, the stocker, and the stocker capacity is assumed to be infinite. Even when a job becomes ready to proceed to the next operation, if no processable machine is immediately available, the job waits in the stocker.

When a machine becomes available, dispatching is performed based on the conditions of the waiting jobs and

the machine. There are two entry points for dispatching: (i) when a job arrives at the stocker and a processable machine is available, it is assigned immediately; and (ii) when a machine becomes idle and waiting jobs exist, a job is selected according to the dispatching algorithm described later.

When a job is assigned to a machine, the system compares the type of the job processed immediately before on that machine with the type of the current job to determine whether setup is required. If setup is required, setup is performed first and processing begins afterward; if setup is not required, processing begins immediately. Equipment failure may occur during setup or processing, and when a failure occurs, the corresponding operation is treated as incomplete. As a result, the interrupted job returns to the stocker in a state requiring the same operation to be performed again, and it then goes through the reassignment process. Interruption and re-release caused by failure may repeat multiple times.

A job that completes all operations is considered completed and is removed from the simulation. When all jobs are completed, the simulation terminates. The time at which the last job is completed and the simulation ends is recorded as the makespan.

2.3.2 Equipment State Changes

figure 2.3.2 shows how equipment states change over time through an example Gantt chart. A machine has one of the

table 2.1.1: Input parameters by equipment group

Group	Number of machines	Processing time range (min)		Changeover setup (min)	Failure distribution parameters			
		P1	P2		Shape k	Scale λ (min)	PM time (min)	Repair time (min)
G1	30	$U(440, 520)$	$U(470, 490)$	120	3.0	11000	80	220
G2	4	$U(170, 190)$	$U(158, 162)$	40	2.8	11000	200	600
G3	15	$U(440, 460)$	$U(390, 410)$	120	2.5	11000	50	150
G4	8	$U(325, 375)$	$U(325, 375)$	90	2.2	11000	200	700
G5	3	$U(145, 155)$	$U(145, 155)$	50	3.3	11000	70	200

following states: idle, setup, processing, repair, or PM, and it processes only one job at a time.

The failure occurrence time of each machine is determined by sampling the time to the next failure from the machine-specific failure distribution (Weibull). When that time elapses, the machine transitions to the failure state, and this can occur not only during the idle state but also during setup or processing. If a failure occurs during processing, the operation in progress is interrupted, and the machine enters the repair state for a predetermined repair time before returning to the available state. When repair is completed, the setup decision criterion of the machine is reset, so setup time does not occur for the first job processed immediately after return.

Preventive maintenance (PM) is performed according to the PM rule described later to reduce the probability of equipment failure. PM is performed only when the machine is idle; if a job is in progress at the scheduled PM time, PM waits until the corresponding operation is completed and is then performed. It is assumed that failures do not occur during PM. When PM is completed, the machine returns to the available state, and as after repair, the next failure time and setup decision criterion are reset, so setup does not occur for the first job after return.

2.3.3 Q-time Violation

figure 2.3.3 shows the Q-time accumulation rule and the method for calculating Q-time violation. Q-time is the maximum allowable waiting time from the completion of one operation to the start of normal processing of the next operation. When a job exceeds this limit, Q-time violation occurs.

Q-time is accumulated from the completion of the previous operation until the start of normal processing of the next operation, and it includes not only waiting in the stocker and setup but also processing time that was interrupted by failure and therefore not completed.

When a job exceeds the Q-time limit, the excess time is recorded as Q-time violation time. At the end of the simulation, the average value is calculated by dividing the total Q-time violation time by the total number of jobs.

2.4 Rule-based algorithm

This section discusses scheduling based on dispatching rules. Six basic dispatching rules that select jobs using a single criterion are defined as baselines, followed by a Composite rule that combines multiple criteria. Both methods are rules that select one candidate job from the stocker when a machine becomes idle, and they select jobs in real time as the simulation proceeds rather than fixing the entire schedule in advance.

2.4.1 Basic Dispatching Rules

table 2.4.1 shows the six basic dispatching rules used in this study and the criteria by which each rule selects a job. Each rule selects one candidate job that can be processed by the current operation using a single criterion. These six rules serve as comparison baselines for assessing the performance differences when the Composite rule and GA combine processing time, setup, and Q-time.

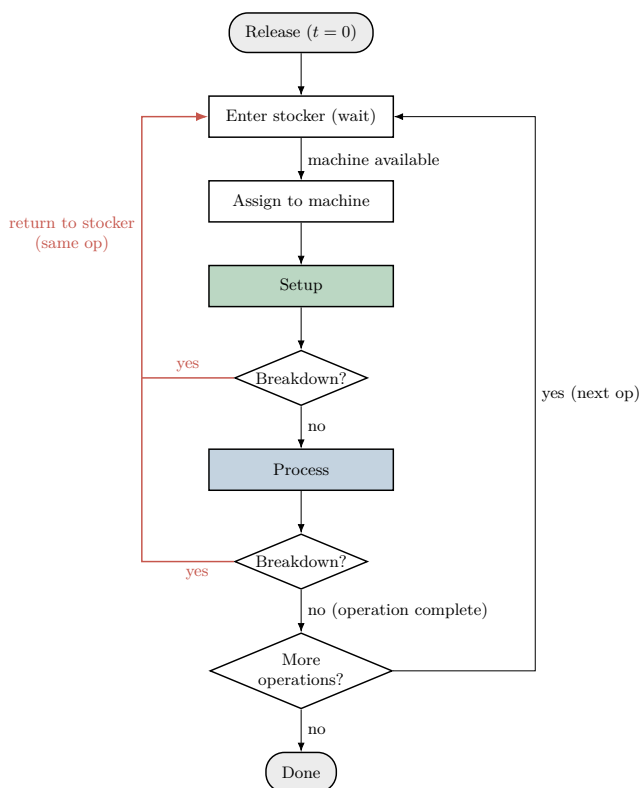


figure 2.3.1: Job flowchart

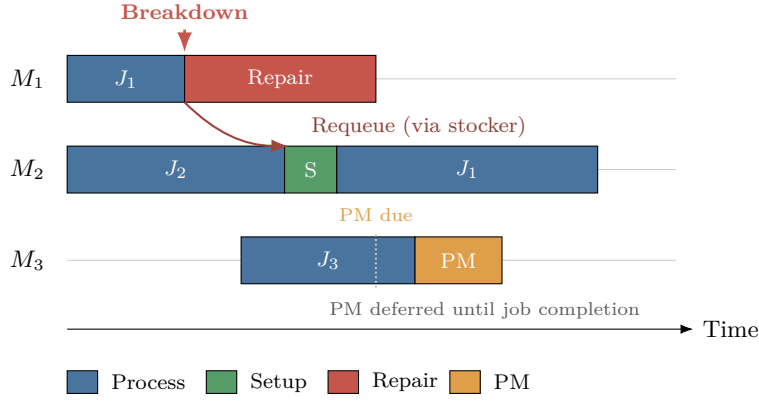


figure 2.3.2: Example Gantt chart

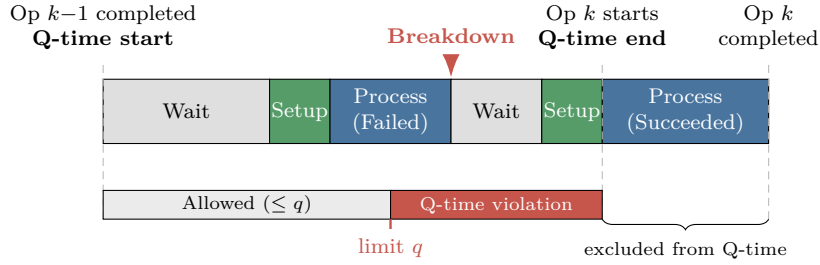


figure 2.3.3: Q-time accumulation rule

table 2.4.1: Basic dispatching rules and selection criteria

Rule	Selected job j^*
random	$j^* \sim \text{Uniform}$
FIFO	$\arg \min_j a_j$
SPT	$\arg \min_j p_{jm}$
LPT	$\arg \max_j p_{jm}$
MIN_QTIME	$\arg \min_j r_j(t)$
SPTSSU	$\arg \min_j (s_{jm} + p_{jm})$

a_j : Stocker arrival time
 p_{jm} : Processing time on machine m
 $r_j(t)$: Remaining Q-time (negative if exceeded)
 s_{jm} : Setup time

Each dispatching rule determines job priority based on different information. SPT and LPT use processing time, MIN_QTIME uses remaining Q-time, and SPTSSU uses a major single criterion such as processing time including setup. Thus, each rule provides a meaningful decision criterion from a specific perspective, but it does not simultaneously consider multiple operational factors in a single dispatching decision.

2.4.2 Composite Rule

The Composite rule does not sort candidate jobs by a single criterion. Instead, it combines processing time, Q-time

slack, setup burden, and operation progress into one score for comparison. For each candidate job j , each priority term $T_k(j)$ is calculated and normalized by dividing it by the empirical maximum value $T_k^{\max}$ of the term ($\hat{T}_k(j) = T_k(j)/T_k^{\max}$). The signs of all terms are aligned so that a smaller value is selected earlier. The final score is as follows.

$$\pi(j) = \sum_{k \in S} w_k \hat{T}_k(j), \quad j^* = \arg \min_j \pi(j)$$

Here, S is the set of terms included in the Composite rule, and w_k is the weight of each term. The six candidate terms reflect different scheduling aspects, as shown in [table 2.4.2](#).

table 2.4.2: Priority terms of the Composite rule

Term	Meaning
PT	Processing time on the current machine
SLACK	Remaining Q-time slack (lower priority if unconstrained)
SETUP	Setup time for product-type changeover
C_TRANSITION	Q-time risk of transition to the next process group
COMPLETION_FAST	Operation progress rate (higher is prioritized)
WAITING	Accumulated waiting time (longer is prioritized)

The weights are not specified manually but are searched.

All six terms are used as candidates, and Particle Swarm Optimization (PSO) is used on a simplex where the sum of weights equals 1 to search for a weight vector that lowers the objective function J . During the search stage, each candidate weight vector is evaluated by the average J from repeated simulations with multiple seeds, and the final performance of the selected weights is compared according to the repeated-run criteria in the performance evaluation method section.

The overall weight search procedure is shown in Algorithm 1.

Algorithm 1. Composite dispatching-rule weight optimization with simplex-PSO

Input: $N, T_{MAX}, D, \omega, c_1, c_2$

Output: w^*

```

1  Initialization:
2  for each particle  $i = 1, \dots, N$  do
3    Sample position  $x_i \sim U([0, 1])$ 
4    Initialize velocity  $v_i \leftarrow 0$ 
5  end for
6  for each particle  $i = 1, \dots, N$  do
7     $pbest_i \leftarrow x_i, F(pbest_i) \leftarrow F(x_i)$ 
8  end for
9   $gbest \leftarrow \arg \min_i F(pbest_i)$ 
10  $t \leftarrow 1$ 
11 Update:
12 while  $t \leq T_{MAX}$  do
13   for each particle  $i = 1, \dots, N$  do
14     Pick random numbers:  $r_1, r_2 \sim U(0, 1)^D$ 
15     Update velocity  $v_i \leftarrow \omega v_i + c_1 r_1 (pbest_i - x_i) + c_2 r_2 (gbest - x_i)$ 
16     Update position  $x_i \leftarrow x_i + v_i$ 
17     Project onto simplex
18      $x_i \leftarrow \max(x_i, 0) / \sum_d \max(x_{i,d}, 0)$ 
19   end for
20   for each particle  $i = 1, \dots, N$  do
21     If  $F(x_i) < F(pbest_i)$  then  $pbest_i \leftarrow x_i, F(pbest_i) \leftarrow F(x_i)$ 
22   end for
23    $gbest \leftarrow \arg \min_i F(pbest_i)$ 
24    $t \leftarrow t + 1$ 
25 end while
26 return  $w^* \leftarrow gbest$ 

```

The weight search was conducted under the condition that PM was disabled and failures were enabled. Therefore, the weights of the Composite rule were adjusted to the dispatching priority, and the effect when combined with PM is examined in a separate experiment.

2.5 PM Rule

The PM rule determines when each machine should be inspected proactively. This study adopts a threshold rule and also considers the age replacement rule as a comparison baseline. Both rules model the cumulative failure probability of equipment using a Weibull distribution, defined for the elapsed time t since the last PM or repair as follows.

$$F(t) = 1 - \exp\left(-\left(\frac{t}{\lambda}\right)^k\right)$$

Here, k is the shape parameter and λ is the scale parameter. In this experiment, $\lambda = 11,000$ min is applied to all machines, and the shape parameters are set by group as $k = 3.0, 2.8, 2.5, 2.2, 3.3$.

2.5.1 Threshold Rule

$$t_{PM} = \lambda (-\ln(1 - \theta))^{1/k}$$

The threshold rule performs PM when the cumulative failure probability $F(t)$ of a machine reaches a pre-specified threshold θ . The smaller θ is, the earlier PM occurs; the larger θ is, the longer the machine is used before PM is performed. Therefore, the threshold rule can be viewed as a PM rule that directly adjusts the level of failure risk allowed.

2.5.2 Age Replacement Rule

$$g(t) = \frac{t_p [1 - F(t)] + t_f F(t)}{\int_0^t [1 - F(u)] du}$$

$g(t)$ Expected downtime per unit operating time at PM interval t

t Elapsed time since the last PM or repair and candidate PM interval

t_p PM duration

t_f Repair time after failure

$F(t)$ Cumulative failure probability up to elapsed time t

$\int_0^t [1 - F(u)] du$ Expected operating time before PM or failure

Based on the age replacement model (Barlow & Proschan, 1965), the age replacement rule selects the PM interval t that minimizes $g(t)$. While the threshold rule determines the PM timing based on the threshold of cumulative failure probability, the age replacement rule uses the balance between planned downtime caused by PM and repair loss after failure as its criterion.

2.6 Genetic Algorithm (GA)

This section describes the detailed design of the GA-based approach introduced in the introduction. In manufacturing process scheduling, GA has been used to jointly explore job sequences and resource assignments in a single solution representation (Kim et al., 2021; Pezzella et al., 2008), and in semiconductor Q-time-constrained scheduling, the design of chromosome encoding and decoding procedures

is treated as an important element (Park & Huh, 2026). The GA in this study represents job release order, machine assignment by operation, and preventive maintenance (PM) criteria by machine in a single chromosome, and repeatedly performs selection, crossover, and mutation to search for better combinations.

2.6.1 Chromosome Encoding

The chromosome in this study consists of three parts: *job_seq*, *machine*, and *pm*. *job_seq* is a permutation representing the job release order, and when multiple jobs are selectable, jobs are selected according to this order. *machine* represents the machine on which each operation will be performed as an index in the list of eligible machines. Therefore, the chromosome can reflect a structure in which one operation can be assigned to one of multiple machines. *pm* specifies one of the predefined candidate values for the preventive maintenance criterion of each machine. In this study, the PM criterion candidates are set to $\{0.05, 0.10, 0.20, 0.35, 0.50, 0.70, 0.90, 1.00\}$, and each value is used as the criterion for performing PM when the Weibull cumulative failure probability of the machine reaches that level.

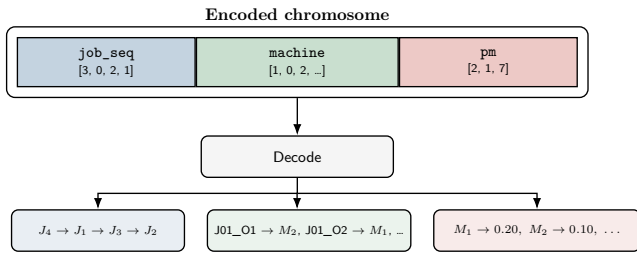


figure 2.6.1: Example of GA chromosome encoding and decoding

figure 2.6.1 shows an example in which a chromosome expressed as integer indices is converted through decoding into job release order, machine assignment by operation, and PM criteria by machine. Equipment failure is not encoded as a decision variable but is reflected as a state change during the simulation evaluation process.

2.6.2 Evaluation and Generation Update

figure 2.6.2 illustrates the generation update process of the GA. The initial population is generated randomly. *job_seq* is generated as a permutation in which every job appears once, machine selects one machine among the eligible machines for each operation, and *pm* selects one PM criterion candidate for each machine. The generated chromosomes

are decoded and evaluated through simulation, and the results are used to determine the relative quality of the solutions.

In each generation, a subset of chromosomes with superior evaluation results is preserved as elites and passed unchanged to the next generation. The remaining individuals are generated as new offspring chromosomes through crossover and mutation after parent chromosomes are selected by tournament selection. The next-generation population is then formed by combining the elites and offspring chromosomes, and the same process is repeated until the specified number of generations is reached. This method preserves high-quality solutions found so far while also exploring new combinations.

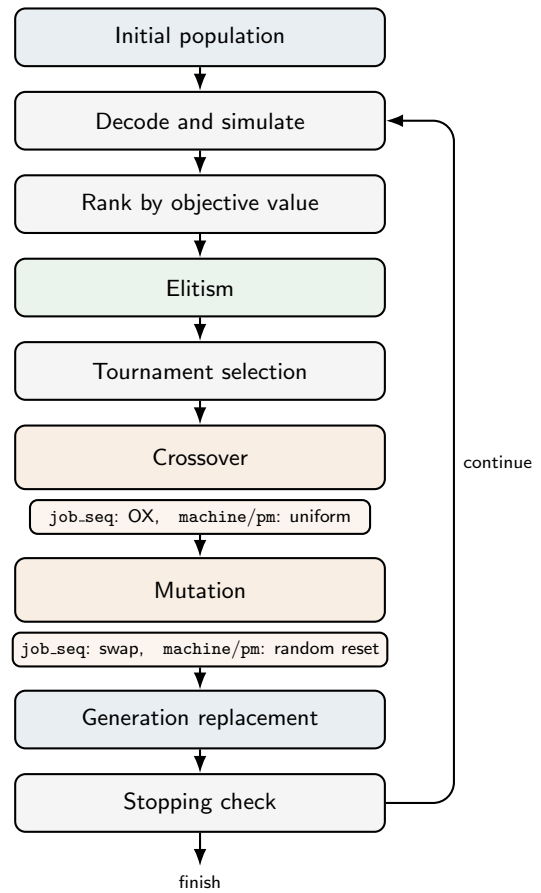
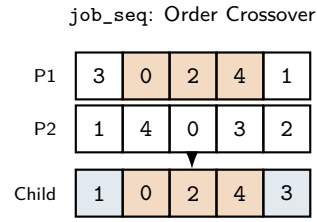


figure 2.6.2: GA generation update flow

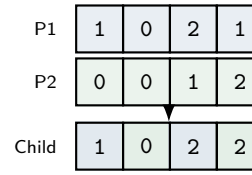
2.6.3 Crossover and Mutation

Crossover and mutation are applied differently to preserve the structure of the three chromosome parts. As shown in figure 2.6.3, *job_seq* must maintain a permutation structure, so order crossover is used to prevent job duplication or omission. For machine and *pm*, since the value at each position is an independent index, uniform crossover is applied. For mutation, swap mutation, which exchanges val-

Crossover



machine/pm: Uniform



Mutation

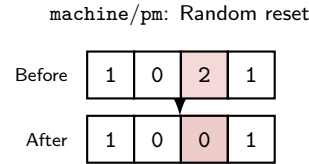
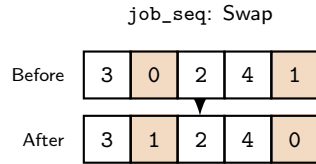


figure 2.6.3: Examples of crossover and mutation operations

ues at two positions, is applied to job_seq; random reset, which changes an existing value to another feasible candidate value, is applied to machine and pm. Through these operations, chromosomes in all generations are kept as executable simulation inputs.

2.7 Performance Evaluation Method

The final performance comparison uses the sample mean of 30 independent repeated runs with different seeds. In the weight search stage of the Composite rule, considering computational cost, each candidate weight is evaluated by the average J from 10 repeated simulations.

The normalized objective function J defined in the objective function section is used as the comparison criterion among policies, and the number of failures during processing is also presented as an auxiliary indicator for interpreting the effect of PM.

3 Results and Analysis

This section summarizes the weight search results of the Composite rule and compares it with basic dispatching rules under the condition without PM. It then examines the effect of applying PM to the Composite rule and the search results of GA, followed by a comparison between the rule-based Composite+PM approach and GA.

3.1 Composite Rule Weights

table 3.1.1: Priority term weights derived from PSO search

Term	Weight
PT	0.000
SLACK	0.432
SETUP	0.408
C_TRANSITION	0.065
COMPLETION_FAST	0.043
WAITING	0.052

The sum of the weights for SLACK and SETUP is approximately 0.84, showing that Q-time slack and setup time are the key terms driving dispatching performance in this environment. The weight of PT is 0, which means that considering setup and Q-time slack together contributed more directly to improving the objective function than a processing-time-only criterion. The remaining terms (C_TRANSITION, COMPLETION_FAST, WAITING) operate as auxiliary factors. PSO was set to a swarm size of 24 and 40 iterations, and each candidate weight was evaluated by the average J from 10 repeated simulations during the search stage.

3.2 Comparison of Basic Dispatching Rules and the Composite Rule (Without PM)

table 3.2.1 shows the effect of the job selection criterion itself on performance under the condition without PM.

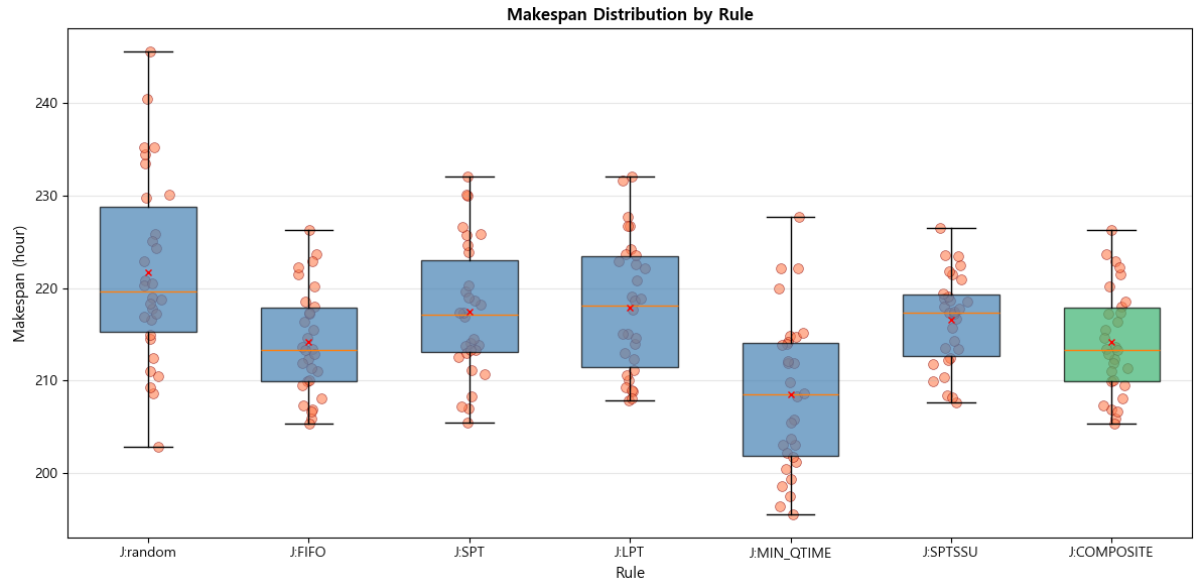


figure 3.2.1: Makespan distribution by rule (without PM)

table 3.2.1: Performance of basic dispatching rules and the Composite rule (without PM)

Rule	Makespan (hour)	AQT (hour)	J
RANDOM	222.30	10.261	1.4518
FIFO	214.73	11.662	1.4933
SPT	216.94	8.325	1.3291
LPT	218.90	4.274	1.1268
MIN_QTIME	211.15	1.947	0.9739
SPTSSU	215.77	14.185	1.6286
COMPOSITE	210.49	1.232	0.9223

Single rules showed different performance depending on the criterion they emphasized. MIN_QTIME directly reduced Q-time violation and achieved the lowest J (0.974) among the single rules, while SPTSSU considered setup but did not reflect Q-time slack, resulting in the largest AQT. The Composite rule recorded a makespan of 210.49, AQT of 1.232, and J of 0.9223, which was lower than all single rules. This can be interpreted as the result of simultaneously reflecting Q-time slack and setup, while combining transition risk and waiting as auxiliary terms. Accordingly, the Composite rule is used as the baseline dispatching rule in the subsequent PM experiments.

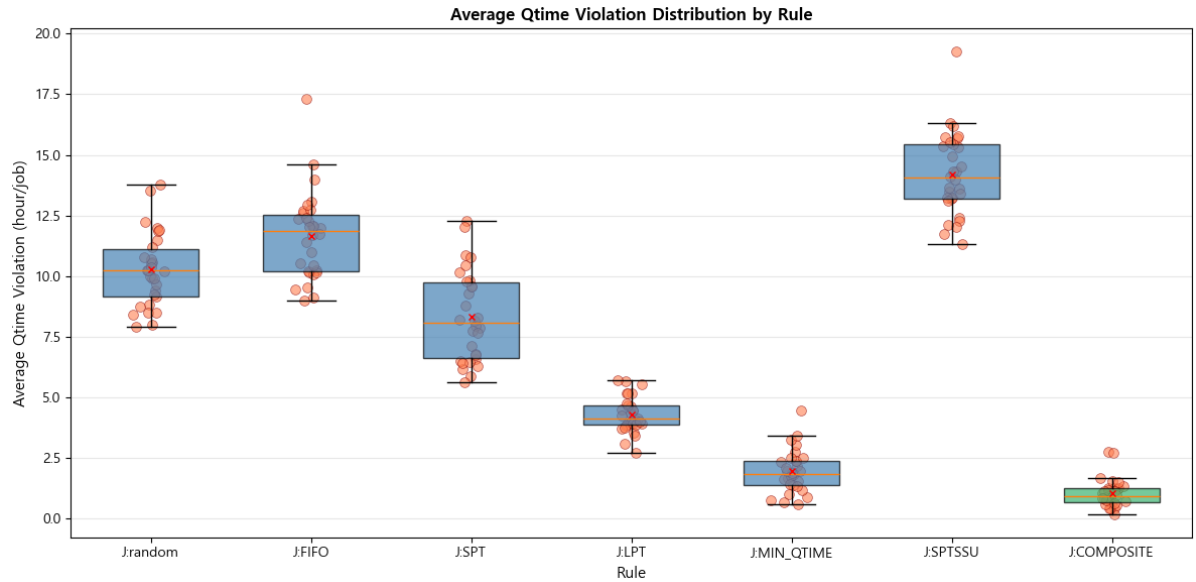


figure 3.2.2: Q-time violation distribution by rule (without PM)

3.3 Applying PM to the Composite Rule

The PM threshold rule was applied to the Composite rule, which achieved the lowest J in the comparison without PM, and makespan, AQT, number of failures during processing, and J were compared while varying the threshold.

whereas the age policy reduced failures further but increased makespan, leading to a higher J . In other words, the PM rule adjusts the trade-off between failure reduction and planned downtime.

table 3.3.1: Performance by PM threshold (Composite)

PM threshold	Makespan (hour)	AQT (hour)	Failures during processing	J
0.05	301.586	5.184	5.70	1.501
0.10	250.285	1.240	7.67	1.084
0.15	235.357	1.385	9.40	1.031
0.30	218.242	1.189	14.20	0.951
0.50	210.191	1.205	18.77	0.919

table 3.3.2: Performance comparison by PM policy (Composite)

Policy	Makespan (hour)	AQT (hour)	Failures during processing	J
PM off	210.49	1.232	20.43	0.9223
PM threshold	210.19	1.205	18.77	0.9192
PM age	220.56	1.248	13.20	0.9641

When the threshold is low (0.05), PM occurs frequently, planned downtime increases, and makespan grows to 301. Conversely, as the threshold increases, the number of PM events decreases, but failures during processing increase from 5.70 to 18.77. In terms of J , the threshold of 0.5 produced the lowest value, 0.919. In the policy comparison, the threshold policy reduced the number of failures while maintaining a makespan and J similar to PM off,

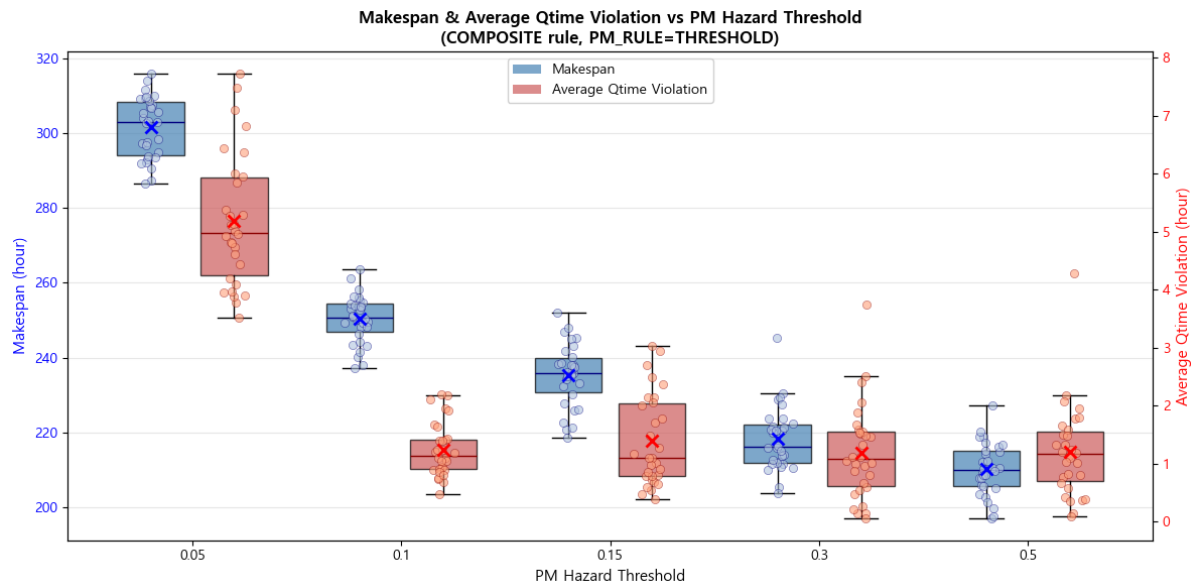


figure 3.3.1: Makespan and average Q-time violation by PM threshold (Composite, threshold rule)

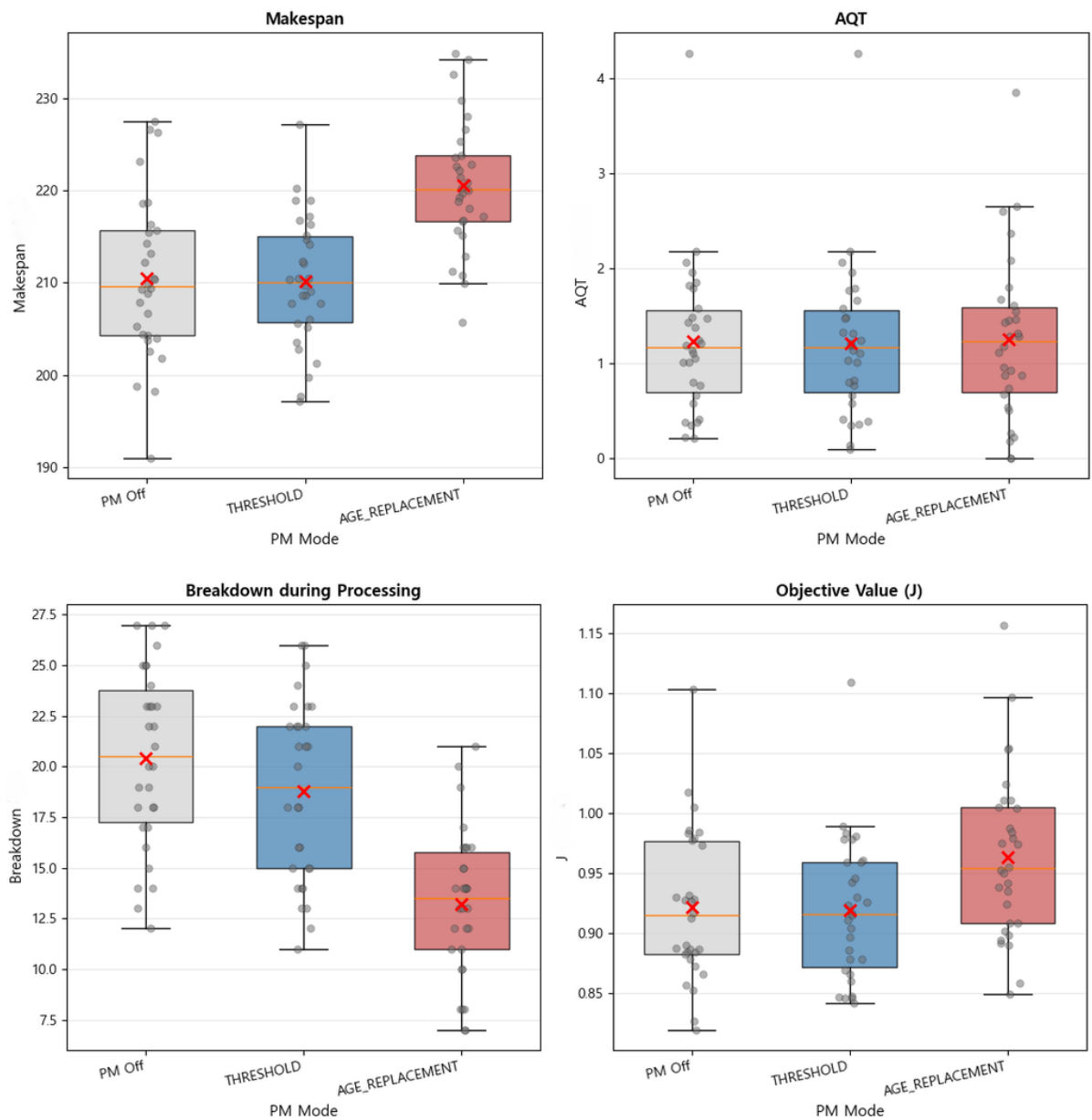


figure 3.3.2: Performance distribution by PM policy (Composite, PM off/threshold/age)

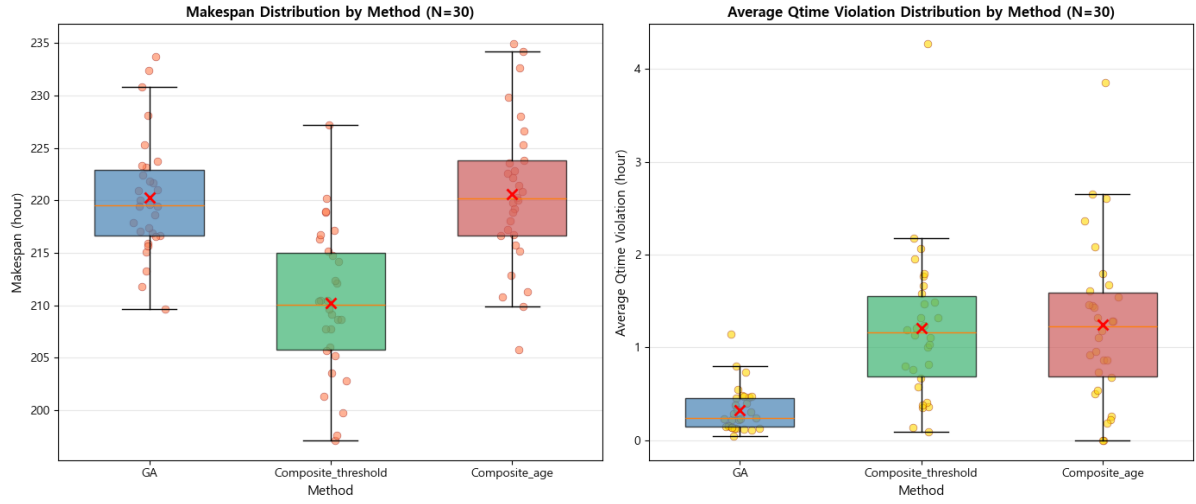


figure 3.5.1: Distribution of makespan and average Q-time violation by method (GA vs Composite+PM)

3.4 GA Results

table 3.4.1: GA execution parameters

Parameter	Value
Population size	100
Generations	1000
Crossover rate	0.8
Mutation rate (job_seq)	0.01
Mutation rate (machine)	0.0005
Mutation rate (PM)	0.005
Selection	Tournament ($k = 3$)
Elitism	1 chromosome

table 3.4.2: Statistics from 30 independent GA runs

Metric	Mean	Standard deviation	Minimum
Best makespan (hour)	220.292	5.672	209.666
Best AQT (hour)	0.328	0.243	0.048
Best J	0.914	0.029	0.860
Failures during processing	8.100	2.578	3.000

GA was run independently 30 times with different seeds while fixing the settings in [table 3.4.1](#). In each run, the chromosome with the lowest objective function value among the results explored up to the final generation was used as the representative solution for that seed. [table 3.4.2](#) summarizes the performance distribution of the 30 representative solutions obtained in this way.

The average J over the 30 runs was 0.914, and the minimum value was 0.860. The average makespan was 220.292 hours, which was not shorter than that of the Composite rule, but the average AQT was low at 0.328 hours. Thus, GA showed a relative strength in mitigating Q-time violation rather than reducing production completion time. This can be attributed to the fact that job sequence, machine assignment, and PM threshold were jointly adjusted within a single chromosome. However, AQT ranged from 0.048 to 1.141 hours and failures during processing ranged from 3 to 14, indicating that variability remained depending on the seed and stochastic failure conditions.

3.5 Comparison of Composite+PM and GA

Finally, the rule-based approach combining the Composite rule with PM was compared with GA ([table 3.5.1](#)).

table 3.5.1: Final comparison of Composite+PM and GA

Method	Makespan (hour)	AQT (hour)	Failures during processing	J
C+PM(threshold)	210.19	1.2046	18.7	0.9193
C+PM(age)	220.56	1.2479	13.2	0.9638
GA	220.29	0.3280	8.1	0.9143

GA achieved a J of 0.914, slightly lower than the 0.919 of Composite+PM(threshold), and in particular reduced AQT to 0.328 hours. This appears to be because it jointly explored job sequences, machine assignments, and PM thresholds. However, the execution time of GA was approximately 366 hours per run, showing a large difference from the approximately 0.01 hours required by Composite+PM.

Composite+PM and GA do not simply replace each other; rather, they have different roles. Composite+PM has short computation time and clear decision criteria, so it can be used as a fast and explainable rule-based alternative in a dynamic production environment. In contrast, GA searches for better schedules through repeated solution exploration and simulation evaluation, so it can be used as a pre-optimization method or comparison baseline for improving solution quality when sufficient computation time is available, rather than as a real-time dispatching rule.

4 Conclusion

For semiconductor process scheduling in which Q-time constraints, preventive maintenance, and stochastic equipment failures act simultaneously, this study compared basic dispatching rules, a Composite rule, a PM threshold rule, and GA in a SimPy-based discrete-event simulation. The main results show not the absolute superiority of a specific method, but rather that each method has a different decision-making scope and computational cost.

Although single dispatching rules have the advantages of interpretability and fast computation, they emphasize only one criterion and therefore showed limitations in the comprehensive objective function of an environment where Q-time, setup, waiting, and transition risk act together. The Composite rule combined multiple priority terms centered on Q-time slack and setup and achieved a lower J than all single rules, showing that reflecting multiple pieces of state information together is more suitable than relying on a single criterion in this environment.

When the PM threshold was combined with the Composite rule, the number of failures could be reduced while maintaining J at a level similar to that of the Composite rule without PM. This shows that PM is not merely a maintenance condition, but a scheduling element that adjusts the trade-off between planned downtime and failure risk.

GA jointly explored job sequences, machine assignments, and PM thresholds, reducing Q-time violation and failures during processing, and it also produced a slightly lower average J than Composite+PM (0.9143 vs. 0.9193). However, it required approximately 366 hours per run, resulting in a very large computational cost, and it had the limitation that the decision structure of the selected solution was difficult to explain.

Therefore, in actual operations, when computation time is limited and the basis for decisions is required, Composite+PM can be a practical alternative. In contrast, when sufficient computation time is available and reducing Q-time violation is important, GA can be used as a pre-optimization method or comparison baseline. In other words, the two methods should be selected according to operational objectives and available computation time rather than being judged solely by performance values.

4.1 Research Significance

The significance of this study lies not merely in comparing the performance of specific algorithms, but in integrating multiple operational conditions that occur simultaneously in actual semiconductor manufacturing environments into a single optimization problem. While many existing studies have addressed the FJSP structure, Q-time constraints, preventive maintenance, and equipment failures separately, this study jointly reflects quality constraints caused by Q-time violation, planned downtime caused by PM, and availability changes caused by stochastic equipment failures. Therefore, the results of this study are meaningful in that they explore the balance between production efficiency and quality constraint management under complex and uncertain conditions closer to actual FAB operations, rather than in an idealized static scheduling environment.

4.2 Limitations and Future Research

The simulation in this study reflected processing time, setup, failure, repair, PM, and Q-time by machine, but it did not include lot transfer time, buffer capacity, chamber-level constraints, batch processing, operator constraints, or rework policies. Since these elements may change

dispatching results in an actual FAB, future work can extend the validation toward a more realistic environment by adding AMR/OHT logistics resources, transfer times between machines, and chamber-level constraints.

The objective function combined makespan and AQT after normalization, but in actual environments, the impact of Q-time violation varies depending on the process stage, product, excess time, rework feasibility, and scrap cost. If Q-time violation is extended from a simple time penalty to a quality cost or loss function, and scrap, rework, PM, repair, and due-date delay costs are reflected together, PM thresholds can be evaluated not as a performance indicator but as an economic criterion.

The PM threshold was evaluated by searching for the value that minimized the objective function, and the changing trends of makespan, AQT, and failures during processing were also examined. However, economic and risk factors of PM, such as the risk cost of failures during processing, machine criticality, spare capacity, replacement part cost, and engineer availability, were not explicitly modeled. Including these factors as cost terms would allow PM decisions to be handled more realistically.

The weights of the Composite rule and the behavior of GA were determined according to the current experimental conditions. If job mix, number of machines, Q-time tightness, setup structure, and failure frequency change, the optimal weights may also change. GA is also sensitive to population size, generation count, crossover/mutation rate, and fitness weights. Therefore, an adaptive rule that dynamically adjusts Composite weights and PM thresholds according to WIP level, equipment utilization, and failure rate, as well as a systematic sensitivity analysis of GA parameters, is needed.

5 References

Al-Hinai, N., & ElMekkawy, T. Y. (2011). Robust and stable flexible job shop scheduling with random machine breakdowns using a hybrid genetic algorithm. *International Journal of Production Economics*, 132(2), 279–291. <https://doi.org/10.1016/j.ijpe.2011.04.020>

Barlow, R. E., & Proschan, F. (1965). *Mathematical theory of reliability*. John Wiley & Sons.

Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 41, 157–183. <https://doi.org/10.1007/BF02023073>

Cassady, C. R., & Kutanoglu, E. (2005). Integrating preventive maintenance planning and production scheduling for a single machine. *IEEE Transactions on Relia-*

bility, 54(2), 304–309. <https://doi.org/10.1109/TR.2005.845967>

Cho, L., Park, H. M., Ryan, J. K., Sharkey, T. C., Jung, C., & Pabst, D. (2014). Production scheduling with queue-time constraints: Alternative formulations. *IIE Annual Conference and Expo 2014*, 282–291.

Kim, S., Park, S., Yoon, Y.-S., & Oh, H. (2021). A study on the scheduling of dyeing processing production based on genetic algorithm. *The Journal of Korean Institute of Communications and Information Sciences*, 46(1), 192–200. <https://doi.org/10.7840/kics.2021.46.1.192>

Park, I. B., & Huh, J. (2026). A genetic algorithm with feasibility-agnostic encoding and three-phase decoding for scheduling semiconductor manufacturing facilities under queue time limits. *Expert Systems with Applications*, 301, 130310. <https://doi.org/10.1016/j.eswa.2025.130310>

Pezzella, F., Morganti, G., & Ciaschetti, G. (2008). A genetic algorithm for the flexible job-shop scheduling problem. *Computers & Operations Research*, 35(10), 3202–3212. <https://doi.org/10.1016/j.cor.2007.02.014>